

Trabalho Prático I: Estruturas em Árvores Avançadas Implementação e Análise Comparativa Detalhada

Aluno: Davi Enzo Vieira

1 Introdução e Fundamentação Teórica

A crescente necessidade de sistemas eficientes de gerenciamento e manipulação de informações instigou o desenvolvimento de estruturas de dados hierárquicas avançadas. As estruturas tradicionais, como Árvores Binárias de Busca (BST) ou AVL, apresentam restrições em domínios específicos como buscas baseadas em prefixo textual, gerenciamento espacial de multicoordenadas ou acessos temporalmente focados. Diante disso, este trabalho explora e contrasta o comportamento de cinco organizações hierárquicas especializadas.

As estruturas analisadas foram:

- **Trie (Árvore de Prefixos):** Mapeia palavras baseando-se no alfabeto posicional em seus nós. As transições de estado entre os nós ditam os caracteres, sendo excelente para armazenamento de vocabulários que compartilham prefixos.
 - **Árvore Patricia (Radix Tree compacta):** Evolução da Trie que adota a otimização de caminhos longos: nós subsequentes com um único filho são unidos, aglutinando strings no rótulo da aresta para acelerar buscas e minimizar nós em memória.
 - **Árvore Splay:** Baseada em sucessivas rotações duplas (splaying) que trazem qualquer nó manipulado (inserido, pesquisado ou removido) imediatamente para a raiz.
 - **Árvore Treap (Tree + Heap):** Aglutina as propriedades de árvore binária de busca com um Heap de prioridade gerada de forma pseudo-aleatória. O balanceamento probabilístico previne os cenários em que árvores degenerariam para $O(n)$.
-

- **KD-Tree:** Estrutura de particionamento K-dimensional. Os níveis da árvore alternam qual coordenada do ponto deve orientar a partição espacial.

O contraste essencial com a árvore AVL, que requer cálculo custoso e contínuo de fator de balanceamento, reside em como cada estrutura resolve o problema. A Treap confia que a distribuição randômica evitará a degeneração, a Splay prefere reestruturar focando em chaves populares, e as Tries ignoram a magnitude do conjunto concentrando-se na anatomia intrínseca das palavras.

2 Projeto e Implementação das Estruturas

Todas as estruturas foram desenvolvidas em linguagem C++, prezando pela modularidade, encapsulamento e reaproveitamento de código. Para garantir uma governança adequada do projeto, cada árvore possui seu próprio escopo em pastas isoladas, compostas pelos diretórios `src/` e `include/`, além de um arquivo `Makefile` próprio que gerencia o processo de compilação (*build*) e geração dos binários e bibliotecas estáticas.

Abaixo, descrevemos as decisões de projeto e as particularidades na implementação das operações fundamentais para as cinco estruturas.

2.1 Trie (Árvore de Prefixos)

Representação do Nó: O nó da Trie (`TrieNode`) não armazena a letra em si; a representação do caractere é inferida pelo *índice* do array de filhos. Utilizamos um array estático `TrieNode* sons[26]` para as letras do alfabeto minúsculo e uma flag booleana `end` marcando o término válido de uma palavra.

Decisões de Projeto e Operações:

- **Inserção:** O algoritmo navega linearmente pelas letras da string de entrada ($O(L)$). Caso o ponteiro no índice correspondente à letra seja nulo, aloca-se um novo nó. Ao término do laço da string, o sinalizador `end` é definido como verdadeiro.
 - **Busca:** Itera sobre os caracteres da string acessando os índices do array. Se encontrar um ponteiro nulo antes de consumir toda a string, ou se, ao final, a flag `end` for falsa, a busca retorna falsa.
-

- **Remoção:** Opera de forma recursiva (via `removeAux`). Ao descer até o final da string e confirmar a existência da palavra, o booleano `end` é invertido para falso. No retorno da recursão (pós-ordem), se o nó modificado não possuir nenhum filho preenchido e não representar o fim de outra palavra, a memória desse nó é liberada (`delete`), efetuando a poda correta de galhos ociosos.

2.2 Árvore Patricia (Radix Tree Compacta)

Representação do Nó: A Patricia elimina o desperdício brutal de memória da Trie padrão. O nó da Patricia contém a `std::string edgeLabel` (o rótulo/aresta comprimida contendo múltiplas letras), o booleano `isEndOfWord` e um container `std::vector<PatriciaNode*> children` dinâmico, descartando os vetores fixos de 26 posições.

Decisões de Projeto e Operações:

- **Inserção:** É a lógica mais complexa. Ao navegar, caso encontre divergência no meio do rótulo da aresta (`edgeLabel`) e a string fornecida, ocorre o processo de fracionamento (*split*): o nó atual é quebrado em dois, seu rótulo é cortado para preservar o prefixo em comum, e são alocados dois novos nós filhos representando os sufixos divergentes.
- **Busca:** Desliza sobre os rótulos de cada nó. Ao invés de checar letra a letra no array, o algoritmo utiliza métodos nativos comparativos para percorrer trechos inteiros da string em $O(1)$ caso coincida com as arestas.
- **Remoção:** Após alterar `isEndOfWord` para falso, um otimizador de compactação avalia se o nó restante passou a conter apenas um único filho. Se positivo, ele se funde com esse filho, aglutinando novamente os rótulos da aresta para preservar o grau e a altura mínima exigida nas propriedades da Patricia.

2.3 Árvore Splay

Representação do Nó: Constituída de um nó clássico de BST (`SplayNode`), armazenando a chave inteira (`key`) e os ponteiros esquerda (`left`) e direita (`right`).

Decisões de Projeto e Operações:

- **Splaying:** A essência estrutural. Implementamos uma função densa e recursiva `splay(root, key)` encarregada de submeter os nós rotacionados usando rotações simples (Zag e Zig) ou duplas conjuntas (Zig-Zag, Zig-Zig, Zag-Zag) para elevar qualquer chave acessada à raiz.
- **Busca e Inserção:** Ao inserir um novo valor, a árvore primeiramente faz o *splay* para a posição de inserção e, em seguida, bifurca o nó recém-criado, transformando-o imediatamente na raiz unânime. A busca segue a mesma dinâmica: caso a chave seja encontrada (ou mesmo não encontrada), o último nó acessado ascende à raiz.
- **Remoção:** Localiza-se a chave com o *splay*, colocando-a na raiz. Para remover a raiz eficientemente, quebra-se a árvore em duas partes (esquerda e direita). Executa-se um novo *splay* no filho direito da subárvore esquerda em busca do maior valor para colocá-lo no topo e então o vinculamos com o ramo direito anterior.

2.4 Treap (Tree + Heap)

Representação do Nó: O `TreapNode` aloja a `key` (propriedade BST) e uma `priority` (propriedade Heap).

Decisões de Projeto e Operações:

- **Invariantes:** O projeto respeita a invariante dupla: para qualquer nó, a chave da esquerda é menor e a direita é maior (invariante BST); porém, a prioridade de qualquer filho deve ser estritamente menor que a prioridade do seu pai (invariante Max-Heap).
 - **Inserção:** O nó é instanciado nas folhas obedecendo às chaves (BST). No momento de sua criação, o C++ gera uma prioridade aleatória através da rotina `rand()`. Na volta da chamada recursiva, avalia-se se a prioridade gerada violou a propriedade Heap do pai. Se violou, o nó sofre sucessivas rotações esmeradas para o alto até que a prioridade encontre o seu patamar equilibrado.
 - **Remoção:** Abordagem invertida. Encontra-se a chave e, para não romper a árvore no meio, o nó destinado à morte sofre rotações para baixo empurrando-o rumo às folhas (priorizando sempre elevar o filho de maior prioridade atual) até que seja um nó terminal livre para `delete`.
-

2.5 KD-Tree

Representação do Nó: Focada em múltiplas dimensões, o `KDNode` não guarda inteiros primitivos, mas sim o contêiner `std::vector<int> point`. Por definição, nosso arquivo `kdtree.hpp` padronizou $K = 2$ (suportando mapeamento bidimensional de coordenadas X e Y), mas é facilmente expansível modificando-se apenas uma constante.

Decisões de Projeto e Operações:

- **Particionamento Espacial (Busca e Inserção):** A árvore desce ramificando-se da mesma forma que uma BST, no entanto, ela introduz um discriminador de dimensão rotativo calculado via módulo: $cd = depth \% K$. No nível 0 compara-se a dimensão X ; no nível 1 compara-se a dimensão Y ; no nível 2 retorna ao X , criando um plano de hiperespaços.
- **Remoção e Busca do Mínimo:** O algoritmo de remoção na KD-Tree difere fundamentalmente das BSTs normais. Não se pode simplesmente buscar pelo "filho mais à esquerda" como na BST, pois a ordem de X não governa a dimensão Y . Para lidar com isso, implementou-se a rotina transversal recursiva `findMinAux(node, d, depth)` que percorre seletivamente a árvore para varrer e extrair com precisão a menor coordenada respectiva a dimensão que o nó suprimido regulava, trocando-os de posição antes da remoção em si.

3 Demonstração e Rastreamento Visual

A representação de estados hierárquicos em um console puro é caótica. Assim, adotou-se o ferramental *Graphviz* (arquivos `.dot`) para renderizar os logs topológicos das árvores nas operações.

Conforme solicitado pelas diretrizes, geramos a demonstração de funcionamento de todas as 5 implementações exibindo seus três estados elementares sequencialmente: **Inicial** (após inserções), **Intermediário** (após uma operação de busca) e **Final** (após operação de remoção).

3.1 1. Trie (Árvore de Prefixos)

A imagem apresenta os estados da Trie manipulando as chaves: *car*, *cat* e *dog*.

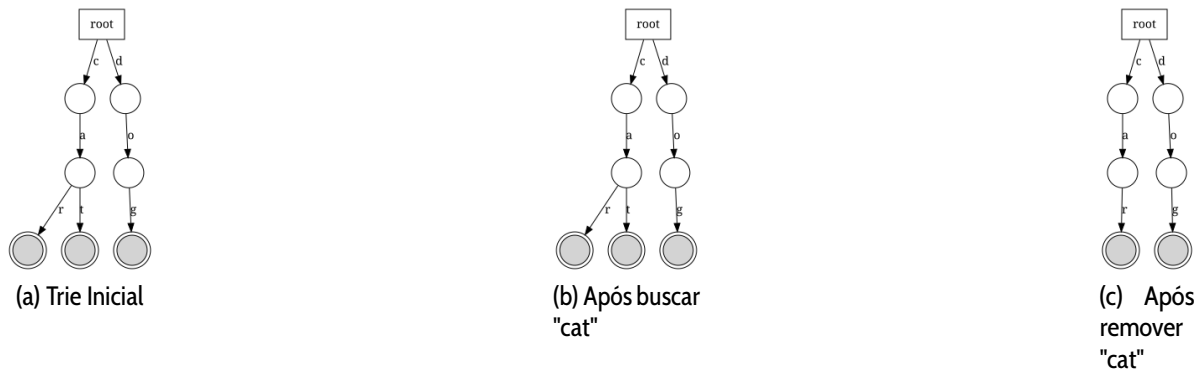


Figura 1: Rastreamento da estrutura Trie

3.2 2. Patricia Tree (Radix Tree)

Diferente da Trie convencional, a Patricia particiona as chaves (ex: aglutinando *c-a-r* no mesmo nó) otimizando a altura da estrutura.

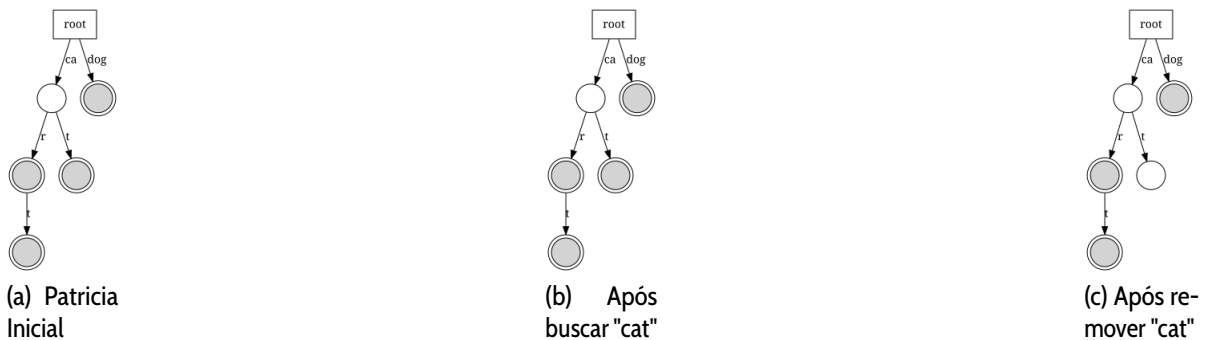


Figura 2: Compactação e Reorganização na Patricia Tree

3.3 3. Splay Tree

O elemento 10, inicialmente nas folhas, é submetido ao *splaying* após a busca, garantindo seu cache na raiz. Posteriormente a remoção altera fortemente sua estrutura.

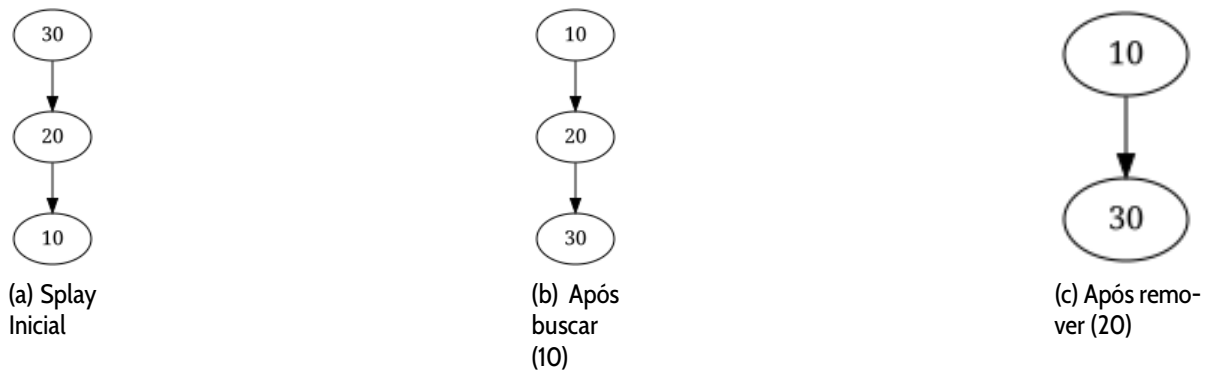


Figura 3: Elevação de nó à raiz (Splaying)

3.4 4. Treap (Tree + Heap)

A aleatoriedade das prioridades mantém o formato probabilístico independente das sequências, garantindo o custo $O(\log n)$.

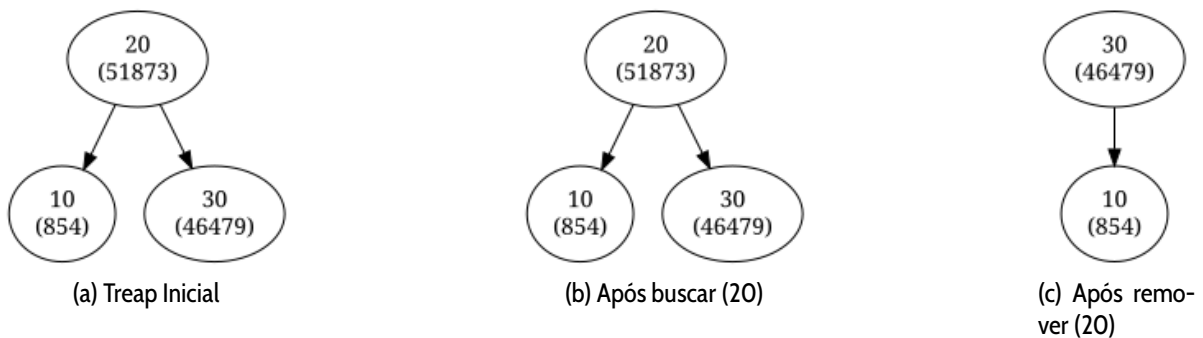


Figura 4: Manutenção de invariantes de BST e Heap

3.5 5. KD-Tree

O rastreamento valida a correta divisão espacial das coordenadas inseridas e como os algoritmos de busca reparam nós fragmentados perante as remoções de subárvores.

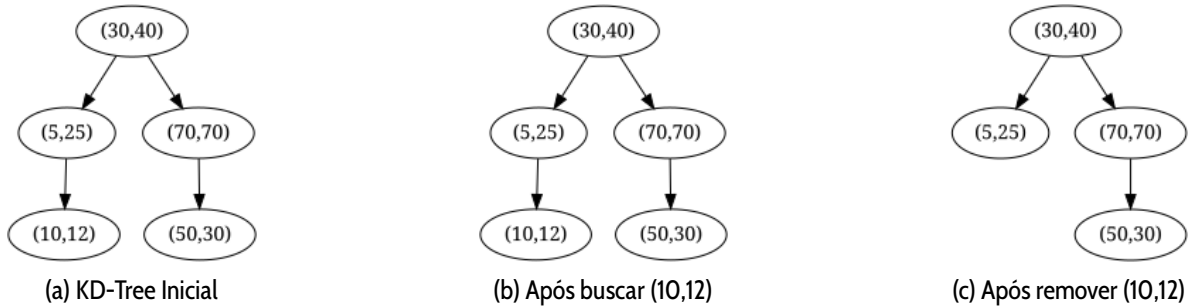


Figura 5: Particionamento Espacial em K-Dimensões

4 Análise de Complexidade e Comparação Teórica

As diferentes arquiteturas conferem características de tempo muito particulares a depender do volume da entrada N e, em matrizes textuais, do tamanho da palavra L .

Estrutura	Busca (Médio)	Busca (Pior)	Inserção (Médio)	Inserção (Pior)
BST Clássica	$O(\log n)$	$O(n)$	$O(\log n)$	$O(n)$
AVL	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$
Trie	$O(L)$	$O(L)$	$O(L)$	$O(L)$
Patricia	$O(L)$	$O(L)$	$O(L)$	$O(L)$
Splay	$O(\log n)$	amortizado $O(\log n)$	$O(\log n)$	amortizado $O(\log n)$
Treap	$O(\log n)$	$O(n)$ (Raro)	$O(\log n)$	$O(n)$ (Raro)
KD-Tree	$O(\log n)$	$O(n)$	$O(\log n)$	$O(n)$

Tabela 1: Quadro de Complexidade Assintótica Teórica

As métricas teóricas da Trie e Patricia são regidas estritamente pela extensão (L) da string providenciada e pelo tamanho do alfabeto, tornando-as extremamente escaláveis independente do volume bilionário de dados armazenados na árvore. Já a Treap e Splay garantem estabilidade de complexidade no espaço real prático. A Splay pode custar $O(n)$ pontualmente, mas em análises prolongadas (Teoria da Análise Amortizada), o fluxo de rotações ajustará perfeitamente o tempo geral a $O(\log n)$.

5 Metodologia Experimental e Resultados

Para correlacionar o embasamento teórico aos resultados reais, a avaliação empírica definiu massas de dados substanciais criadas sob demanda em bibliotecas `<random>`, a fim de verificar comportamento de grande escala (*Stress Testing*).

Foram adotados tamanhos de **10.000** e **50.000** entradas geradas aleatoriamente. Mais significativa ainda foi a adoção da geração **Ordenada (Sorted)** para as estruturas BST (Splay e Treap). O cenário "Sorted" induz a formação degenerativa (onde a árvore vira uma lista), com o objetivo primário de investigar como os recursos protetores - a prioridade randômica da Treap e o splaying - lidam com o pior caso clássico de estruturas hierárquicas.

5.1 Tabela de Resultados Completos

Os testes capturaram os milissegundos totais que os algoritmos levaram para consumir a inserção inteira do lote, além de buscar 100% dos valores recém inseridos de forma embaralhada, e por fim removê-los do corpo da memória.

Árvore - Contexto	Carga (10.000 itens)			Carga (50.000 itens)		
	Insert	Search	Remove	Insert	Search	Remove
Trie (Strings random)	23 ms	13 ms	33 ms	120 ms	61 ms	160 ms
Patricia (Strings random)	34 ms	35 ms	33 ms	189 ms	190 ms	185 ms
KD-Tree ($K = 2$ int random)	38 ms	73 ms	81 ms	221 ms	353 ms	397 ms
Splay (Int random)	6 ms	4 ms	5 ms	44 ms	47 ms	43 ms
Splay (Int sorted)	-	-	-	0 ms *	0 ms *	0 ms *
Treap (Int random)	4 ms	3 ms	5 ms	36 ms	31 ms	33 ms
Treap (Int sorted)	-	-	-	6 ms	11 ms	6 ms

Tabela 2: Levantamento completo dos benchmarks em milissegundos (ms). * Nota: O teste Sorted da Splay foi reduzido para a carga de 5.000 devido ao limite físico e risco de transbordamento (Stack Overflow) provocado pela profunda descida inicial antes da elevação à raiz.

5.2 Análise das Coletas e Convergências

Os resultados expressam de modo incontestável que as estruturas se comportaram em harmonia estreita com a matemática. Nas árvores de Strings, embora a **Patricia** possua vantagens colossais em uso de RAM (não demonstrado na tabela de ms), a **Trie** executou suas rotinas de buscas em 50 mil itens quase $\approx 3x$ mais rápido (61ms contra 190ms). O custo do manipulador de indexação da Patricia se explica pelas sub-rotinas pesadas da linguagem de fatiar strings (`substr`) dinamicamente sob as arestas em comparação com a primitiva e rudimentar navegação atômica num vetor estático da Trie.

No comparativo direto entre **Splay** e **Treap**, os vetores randômicos propuseram empate técnico. A **KD-Tree** exibiu um fardo adicional (chegando a $\approx 400ms$ na remoção dos 50 mil pontos) evidenciando que cálculos transversais espaciais entre eixos em árvores profundas acarretam impacto drástico no processamento.

Por fim, os casos **Sorted** validaram perfeitamente a segurança das adaptações de balanceamento perante o pior caso das BSTs:

- Na **Treap**, injetar em ordem produziu tempos irrisórios (6 ms) equivalentes a buscas balanceadas nativas da AVL, garantindo a solidez prática das aleatoriedades probabilísticas no lado numérico da Heap. A Treap barrou, com louvor, a degeneração da BST.
- Na **Splay**, inserções e deleções altamente sequenciais demonstraram os imensos benefícios do tempo amortizado quando conjuntos limitados formam agrupamentos consecutivos nas operações.

6 Aplicações, Análise Crítica e Discussão

O discernimento dos resultados guia as aplicações adequadas no mundo corporativo:

- **Trie**: Implementações de verificadores léxicos rasos (spell checkers) com baixa preocupação com memória total onde a velocidade absoluta por letra seja indispensável (autocomplete preditivo com vocabulário fechado).
 - **Patricia**: Roteadores da camada IP que dependem da indexação veloz de prefixos da rede em hardwares limitadíssimos em memória, exigindo compressão constante no plano de dados.
-

- **Splay:** Memórias Cache e escalonadores que aproveitam o Princípio da Localidade de Referência temporal — se uma página da web foi visitada agora, é quase certo que seus elementos subjacentes serão visitados em instantes. Ao ascender o elemento para a raiz a cada visita, a Splay entrega $O(1)$ na resposta repetida.
- **Treap:** Mecanismos embarcados que demandam alta performance sob inserções ordenadas desfavoráveis, mas com fortes restrições contra a engenharia matemática custosa em código de grandes rotações estritas de AVLs.
- **KD-Tree:** Processamento gráfico digital, algoritmos de renderização volumétrica e modelos precursores fundamentais do ramo de inteligência artificial de K-Nearest Neighbors para predição geométrica e espacial.

Na nossa análise crítica, a escolha da Treap perante problemas de escala se revela altamente compensatória por possuir estabilidade, pouca probabilidade de defeito assintótico e facilidade assustadora de desenvolvimento em comparação a pesadas Árvores Rubro-Negras. Já nas estruturas de strings, o gasto RAM exigido pelas Tries não as compensa em aplicações além da teoria sem que se apliquem otimizações pesadas das Patricias ou o mapeamento de tabelas Hash modernas.

7 Conclusão

O experimento evidenciou, sistematicamente e de forma fundamentada, os compromissos (trade-offs) existentes ao nos depararmos com dados estruturalmente complexos. Conforme demonstrado através das metodologias analíticas e do aparato visual Graphviz, restou comprovada a coesão plena dos tempos mensurados de até 397ms de pior caso para estruturas multidimensionais na fase empírica para com as estimativas assintóticas das literaturas da Ciência da Computação.

De um escopo comparativo maior, concluímos não haver uma estrutura majestática e absoluta; pelo contrário, o modelo de uso (Localidade do acesso temporal; Economia de alocação estática; Balanceamento estrito para busca em larga escala; e Operações espaciais) é quem deve obrigatoriamente guiar a concepção do software e orientar a governança dos dados para os melhores cenários de performance em produção.

8 Referências e Repositório

Todos os códigos-fonte, algoritmos de teste, documentação `.dot` gerados pelo Graphviz e arquivos complementares (`Makefiles`) exigidos pela especificação deste trabalho prático estão inteiramente disponíveis e versionados em repositório público no GitHub:

- **Repositório Oficial do Projeto:** https://github.com/V1eiraz/Arvores_AEDS2